

SPRINT 3

TAREA 7

Actualizar una tarea

PROYECTO INDIVIDUAL · PLANIFICADOR DE TAREAS

JavaScript · TaskManager · Eventos · DOM · data-* · Arrays

APERTURA DEL SPRINT 3

En este sprint continuaremos evolucionando el mismo Planificador de Tareas construido durante los sprints anteriores. El Sprint 3 estará compuesto por la Tarea 7 · Actualizar una tarea y la Tarea 8 · Persistir tareas en LocalStorage. Cada nueva funcionalidad debe integrarse sobre el código existente, conservando los avances, la estructura y el repositorio del proyecto.

OBJETIVO

Implementar la actualización del estado de una tarea para que, al hacer clic en el botón “Mark As Done”, la tarea seleccionada cambie su propiedad status a “DONE” y la interfaz se vuelva a renderizar con la información actualizada.

RESULTADO ESPERADO

- Cada tarea mostrada en la lista cuenta con un botón “Mark As Done”.
- El botón puede identificarse mediante la clase done-button.
- El click se gestiona mediante un único EventListener asociado a la Lista de Tareas.
- La tarea seleccionada se identifica utilizando su data-task-id y el método getTaskById().
- El status de la tarea seleccionada cambia a “DONE”.
- Después de actualizar el estado, TaskManager vuelve a renderizar la lista.

ANTES DE COMENZAR

Continúa trabajando sobre el mismo proyecto del Planificador de Tareas. La funcionalidad de actualización debe integrarse sobre la estructura y el código que construiste durante los sprints anteriores.

ANTES	AHORA	RESULTADO
Las tareas pueden crearse y mostrarse.	Cada tarea incorpora la acción “Mark As Done”.	El usuario puede iniciar la actualización desde la interfaz.
TaskManager conserva las tareas en this.tasks.	getTaskById(taskId) permite recuperar la tarea seleccionada.	Se identifica exactamente qué objeto debe actualizarse.
La lista se renderiza con los datos existentes.	El click obtiene la tarea y cambia task.status.	La tarjeta vuelve a renderizarse mostrando el nuevo estado.

ESTRUCTURA QUE DEBES CONSERVAR

```
Planificador-Tareas/  
├── css/  
│   └── styles.css  
├── js/  
│   ├── index.js  
│   └── taskManager.js  
├── index.html  
└── README.md
```

RECUERDA

La estructura puede variar si tu proyecto ya incorpora archivos adicionales. Lo importante es conservar la separación entre HTML, estilos y JavaScript, mantener los avances de los sprints anteriores y realizar esta nueva funcionalidad dentro del mismo repositorio.

DESARROLLO DE LA ACTIVIDAD

PASO 1 · AGREGAR EL BOTÓN “MARK AS DONE”

En este paso agregarás un botón dentro de cada tarea para que el usuario pueda marcarla como completada. La clase done-button permitirá identificar posteriormente el botón desde JavaScript.

RECURSOS ÚTILES

- Bootstrap · Buttons
- Estructura HTML de las tarjetas

1. En js/taskManager.js, dentro de la función createTaskHtml, agrega un botón al HTML de la tarea.
2. Asigna al botón la clase done-button. Puedes complementarla con las clases de Bootstrap que ya utiliza tu proyecto.

```
<button class="done-button btn btn-success">  
  Mark As Done  
</button>
```

PRUEBA EL PASO 1

Abre index.html en el navegador y crea una nueva tarea usando el formulario. Ahora deberías ver que cada tarea incluye un botón “Mark As Done”.

PASO 2 · AGREGAR UN EVENT LISTENER A LA LISTA DE TAREAS

No agregarás un EventListener a cada botón “Mark As Done”. En su lugar, utilizarás un único EventListener sobre la Lista de Tareas y recorrerás el DOM para identificar el elemento sobre el que se hizo clic. Esta técnica reduce la cantidad de escuchadores necesarios y aprovecha el recorrido del DOM.

RECURSOS ÚTILES

- EventTarget.addEventListener()
- Document.querySelector
- Event.target
- Element.classList
- Recorrido del DOM
- Node.parentElement

1. Asegúrate de que la Lista de Tareas en index.html tenga un id que puedas utilizar para seleccionarla. Por ejemplo: tasksList.
2. En js/index.js, al final del archivo, usa querySelector para seleccionar la lista de tareas y guárdala en una variable.
3. Agrega un EventListener a la lista de tareas, escuchando el evento click.
4. En el Event Handler, recibe el parámetro event.

```
tasksList.addEventListener('click', (event) => {
  // "event" es el parámetro del evento
});
```

5. Usando event.target y un if, verifica si el classList del elemento contiene la clase done-button.
6. Si la condición se cumple, utiliza recorrido del DOM, por ejemplo mediante parentElement, para encontrar el elemento que representa la tarea. Guarda ese elemento en una variable parentTask.

PRUEBA EL PASO 2

Agrega un console.log(parentTask) justo después de encontrar el elemento. Crea una tarea, haz clic en “Mark As Done” y verifica en la consola que se imprime el elemento correspondiente a la tarea.

PASO 3 · AGREGAR EL ID DE LA TAREA AL DOM

Para actualizar la tarea correcta necesitamos relacionar el elemento HTML seleccionado con el objeto Task correspondiente en TaskManager. Para esto utilizaremos el id único de la tarea como un atributo de datos (data-task-id).

RECURSOS ÚTILES

- Uso de atributos de datos
- Template literals

1. En js/taskManager.js, localiza la función createTaskHtml creada anteriormente.
2. Agrega un nuevo parámetro a la función: id.
3. Dentro del HTML de la tarea, usa un marcador de posición \${} en el template literal para agregar el id como valor del atributo data-task-id del elemento contenedor de la tarea.
4. En js/taskManager.js, localiza el método render() de la clase TaskManager.
5. Encuentra dónde se utiliza createTaskHtml y pasa task.id como el nuevo parámetro id.

```
createTaskHtml(id, name, description, dueDate, status) {
  // HTML de la tarea
  // ...
  // data-task-id="${id}"
}
```

PRUEBA EL PASO 3

Repite la prueba de parentTask. En la consola del navegador, el elemento de la tarea ahora debería mostrar el atributo data-task-id con el id correspondiente.

PASO 4 · AGREGAR `getTaskById()` A TASKMANAGER

Ahora que el id de la tarea está disponible en el DOM, necesitamos una forma de utilizarlo para recuperar el objeto correcto desde `this.tasks`. Para esto implementaremos el método `getTaskById()`.

RECURSOS ÚTILES

- Bucles e iteración

- Recorrido de arreglos

1. En `js/taskManager.js`, localiza la clase `TaskManager`.
2. Agrega un nuevo método `getTaskById()` que reciba `taskId` como parámetro.
3. Crea una variable `foundTask` para almacenar la tarea encontrada. Como su valor cambiará, utiliza `let`.
4. Recorre el arreglo `this.tasks`. Para cada tarea, guarda el elemento actual en una variable llamada `task` y compara `task.id` con `taskId`. Si coinciden, guarda `task` en `foundTask`.
5. Después del recorrido, devuelve `foundTask`.

```
getTaskById(taskId) {
  let foundTask;

  for (let task of this.tasks) {
    if (task.id === taskId) {
      foundTask = task;
    }
  }

  return foundTask;
}
```

PRUEBA EL PASO 4

Agrega una tarea desde el formulario, abre la consola de JavaScript y ejecuta `getTaskById()` usando la instancia de `TaskManager`. Por ejemplo, `taskManager.getTaskById(0)`. Si la primera tarea tiene id 0, deberías obtener el objeto correspondiente.

PASO 5 · ACTUALIZAR EL ESTADO A “DONE”

Ya contamos con las piezas necesarias: el botón identifica la interacción, el DOM contiene el id de la tarea y `TaskManager` puede recuperar el objeto correspondiente. Ahora combinarás estos elementos para cambiar `task.status` a “DONE”.

RECURSOS ÚTILES

- Uso de atributos de datos

- `Number()`

1. En `js/index.js`, localiza el `EventListener` del evento click de la Lista de Tareas.
2. Después de encontrar `parentTask`, crea una variable `taskId` usando el data-attribute `taskId` del elemento DOM.
3. Convierte el valor obtenido a `Number`, ya que los atributos `data-*` se recuperan como `String` y los ids de las tareas se manejan como `Number`.
4. Usando `taskId` como parámetro, llama a `taskManager.getTaskById(taskId)` y guarda el resultado en una variable `task`.
5. Cambia el status de `task` a 'DONE'.
6. Vuelve a renderizar la lista llamando a `taskManager.render()`.

```
const taskId = Number(parentTask.dataset.taskId);
const task = taskManager.getTaskById(taskId);

task.status = 'DONE';
taskManager.render();
```

FLUJO DE LA ACTUALIZACIÓN

Usuario hace clic en “Mark As Done” → se identifica done-button → se encuentra parentTask → se obtiene data-task-id → se convierte el id a Number → `getTaskById(taskId)` recupera la tarea → `task.status` cambia a “DONE” → `taskManager.render()` actualiza la interfaz.

PRUEBAS DE FUNCIONALIDAD

No te limites a comprobar únicamente el caso de una tarea. Verifica que el botón actualice exactamente la tarea sobre la que se hizo clic y que las demás permanezcan sin cambios.

CASO 1 · MARCAR UNA TAREA COMO HECHA

- ✓. Abre la aplicación.
- ✓. Crea una nueva tarea.
- ✓. Haz clic en “Mark As Done”.
- ✓. Comprueba que el estado de la tarea cambia a “DONE”.

CASO 2 · ACTUALIZAR UNA TAREA ESPECÍFICA

- ✓. Crea al menos tres tareas.
- ✓. Haz clic en “Mark As Done” de una de ellas.
- ✓. Comprueba que únicamente la tarea seleccionada cambia su estado.
- ✓. Verifica que las otras tareas mantienen su estado original.

CHECKLIST DE ENTREGA · TAREA 7

LISTA DE VERIFICACIÓN

☐	El botón “Mark As Done” aparece en cada tarea.
☐	El botón tiene la clase done-button.
☐	La Lista de Tareas tiene un id utilizable desde JavaScript.
☐	La Lista de Tareas utiliza un único EventListener para detectar el click.
☐	El elemento de la tarea contiene data-task-id.
☐	createTaskHtml() recibe el id y render() envía task.id.
☐	TaskManager contiene el método getTaskById(taskId).
☐	El data-task-id se convierte de String a Number.
☐	La tarea seleccionada cambia su status a 'DONE'.
☐	Después de actualizar, se ejecuta taskManager.render().
☐	La funcionalidad no rompe las tareas o avances construidos anteriormente.
☐	Los cambios fueron guardados mediante commits en el mismo repositorio.

CONTINUIDAD DEL SPRINT 3

PRÓXIMA TAREA · TAREA 8

En la siguiente actividad se trabajará la persistencia de las tareas en LocalStorage. El objetivo será conservar la información del TaskManager aunque el usuario recargue o vuelva a abrir la aplicación. Esta funcionalidad deberá construirse sobre el mismo repositorio y sobre los avances realizados en la Tarea 7.

REGLA DE TRABAJO DEL SPRINT

Cada tarea del Sprint 3 debe integrarse al proyecto existente. Conserva el historial de commits, evita reemplazar el repositorio por uno nuevo y verifica que las funcionalidades desarrolladas anteriormente continúen funcionando después de cada cambio.